

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.8

CRUD

 Copy as Markdown

CRUD

To interact with the database you need a `Session` object as this object holds the connection to the database. All CRUD operations are accessible via the session object and the generated models. The methods can be found under the static `db` field in your generated models.

For the following examples we will use this model:

```
class: Company
table: company
fields:
  name: String
```

NOTE

You can also access the database methods through the session object under the field `db`. However, this is typically only recommended if you want to do custom queries where you explicitly type out your SQL queries.

Create

There are two ways to create a new row in the database.

Inserting a single row

Inserting a single row to the database is done by calling the `insertRow` method on your generated model. The method will return the entire company object with the `id` field set.

```
var row = Company(name: 'Serverpod');
var company = await Company.db.insertRow(session, row);
```



Ask AI

Inserting several rows

Inserting several rows in a batch operation is done by calling the `insert` method. This is an atomic operation, meaning no entries will be created if any entry fails to be created.

```
var rows = [Company(name: 'Serverpod'), Company(name: 'Google')];  
var companies = await Company.db.insert(session, rows);
```

Ignoring conflicts

When inserting rows that might violate a unique or exclusion constraint, you can set `ignoreConflicts` to `true` on the `insert` method. Rows that would cause a unique or exclusion constraint violation are silently skipped, and only the non-conflicting rows are inserted.

```
var rows = [Company(name: 'Serverpod'), Company(name: 'Google')];  
var inserted = await Company.db.insert(session, rows, ignoreConflicts: true);
```

The method returns only the rows that were successfully inserted. If all rows conflict, an empty list is returned. Unlike a regular `insert`, which fails entirely if any row violates a constraint, `ignoreConflicts` allows partial inserts where only the non-conflicting rows are written.

This is useful for idempotent operations where you want to insert data without failing on duplicates. Only unique and exclusion constraint violations are ignored. Other violations such as `NOT NULL`, `CHECK`, or foreign key constraints still throw an exception.

⚠ WARNING

When using `ignoreConflicts` with models that have `non-persistent fields`, each row is inserted individually instead of in a single batch. This is necessary because the database cannot report which rows were skipped in a batch insert, making it impossible to correctly match non-persistent field values back to inserted rows. For large numbers of rows, this can cause performance issues. Consider removing non-persistent fields from the model or inserting in smaller batches.

Read

There are three different read operations available.

Finding by id

You can retrieve a single row by its `id`.

```
var company = await Company.db.findById(session, companyId);
```

This operation either returns the model or `null`.

Finding a single row

You can find a single row using an expression.

```
var company = await Company.db.findFirstRow(  
  session,  
  where: (t) => t.name.equals('Serverpod'),  
);
```

This operation returns the first model matching the filtering criteria or `null`. See [filter](#) and [sort](#) for all filter operations.

INFO

If you include an `orderBy`, it will be evaluated before the list is reduced. In this case, `findFirstRow()` will return the first entry from the sorted list.

Finding multiple rows

To find multiple rows, use the same principle as for finding a single row.

```
var companies = await Company.db.find(  
  session,  
  where: (t) => t.id < 100,  
  limit: 50,  
);
```

This operation returns a `List` of your models matching the filtering criteria.

See [filter](#) and [sort](#) for all filter and sorting operations and [pagination](#) for how to paginate the result.

Update

There are multiple update operations available for different use cases.

Update a single row

To update a single row, use the `updateRow` method.

```
var company = await Company.db.findById(session, companyId); // Fetched company has its  
id set  
company.name = 'New name';  
var updatedCompany = await Company.db.updateRow(session, company);
```

The object that you update must have its `id` set to a non-`null` value and the id needs to exist on a row in the database. The `updateRow` method returns the updated object.

Update specific columns

It is possible to target one or several columns that you want to mutate, meaning any other column will be left unmodified even if the dart object has introduced a change.

```
var company = await Company.db.findById(session, companyId);
company.name = 'New name';
company.address = 'Baker street';
var updatedCompany = await Company.db.updateRow(session, company, columns: (t) =>
[t.name]);
```

In the above example, only the company name will be updated, the address column will not be changed.

Update several rows

To batch update several rows use the `update` method.

```
var companies = await Company.db.find(session);
companies = companies.map((c) => c.copyWith(name: 'New name')).toList();
var updatedCompanies = await Company.db.update(session, companies);
```

This is an atomic operation, meaning no entries will be updated if any entry fails to be updated. The `update` method returns a `List` of the updated objects.

Update specific columns

The same syntax is available for updating specific columns on multiple rows.

```
var companies = await Company.db.find(session);
companies = companies.map((c) => c.copyWith(name: 'New name', address: 'Baker
Street')).toList();
var updatedCompanies = await Company.db.update(session, companies, columns: (t) =>
[t.name]);
```

Update by ID

To update a row by its ID without fetching it first, use the `updateById` method. This method allows you to specify which columns to update directly.

```
var updatedCompany = await Company.db.updateById(
  session,
  companyId,
```

```
columnValues: (t) => [t.name('New name'), t.address('New address')],
);
```

The `updateById` method updates only the specified columns for the row with the given ID. The method returns the updated row, or throws a `DatabaseUnexpectedResultException` if no row with the given ID exists. At least one column must be specified in the `columnValues` parameter, otherwise an `ArgumentError` will be thrown.

See [Database exceptions](#) for the full set of database exception types and when they are thrown.

You can also update columns to null values:

```
var updatedCompany = await Company.db.updateById(
  session,
  companyId,
  columnValues: (t) => [t.name(null), t.address(null)],
);
```

Update where

To update rows based on filter criteria, use the `updateWhere` method. This method allows you to update specific columns for all rows matching a where clause.

```
var updatedCompanies = await Company.db.updateWhere(
  session,
  columnValues: (t) => [t.name('Updated name')],
  where: (t) => t.name.like('%Ltd'),
);
```

The `updateWhere` method updates all rows matching the where expression, modifying only the specified columns. The method returns a list of the updated rows. If no rows match the criteria, an empty list is returned. See [filter](#) for all available filtering operations.

The method also supports [pagination](#) and [ordering](#):

```
var updatedCompanies = await Company.db.updateWhere(
  session,
  columnValues: (t) => [t.name('Updated name'), t.address('New address')],
  where: (t) => t.id > 100,
  orderBy: (t) => t.id, // or t.id.asc()
  limit: 10,
  offset: 5,
);
```

Upsert

An upsert inserts a row, or updates the existing row when it collides with one that is already stored. The database runs the check and the write as a single atomic operation, so no other transaction can slip in between them. The collision is decided by `conflictColumns`, which must be covered by a unique index (or be the primary key); without one, the call throws a `DatabaseQueryException`. The examples below use a `Product` model whose `sku` field has a `unique index`.

Upsert a single row

To insert-or-update a single row, use the `upsertRow` method:

```
var product = await Product.db.upsertRow(
  session,
  Product(sku: 'chair-01', name: 'Office chair', price: 199.00),
  conflictColumns: (t) => [t.sku],
);
```

If no row with that `sku` exists, the row is inserted. If one exists, it keeps its `id`, and every persistent column except `id` and the `conflictColumns` is overwritten with the supplied values. The method returns the stored row.

The `id` column can be the conflict target as well, which is useful when the same code path handles an object that may or may not already have an id:

```
var product = await Product.db.upsertRow(
  session,
  Product(id: productId, sku: 'chair-01', name: 'Office chair', price: 199.00),
  conflictColumns: (t) => [t.id],
);
```

To limit which columns an update touches, pass `updateColumns`. A new row is still inserted with all of its values, and an existing row only has the selected columns updated:

```
var product = await Product.db.upsertRow(
  session,
  Product(sku: 'chair-01', name: 'Renamed chair', price: 249.00),
  conflictColumns: (t) => [t.sku],
  updateColumns: (t) => [t.price],
);
```

To update only rows in a certain state, pass `updateWhere`. When the existing row does not match the expression, nothing is changed and `upsertRow` returns `null`:

```
var product = await Product.db.upsertRow(
  session,
  Product(sku: 'chair-01', name: 'Office chair v2', price: 249.00),
```

```
conflictColumns: (t) => [t.sk],
updateWhere: (t) => t.price > 500.0,
);
```

INFO

The literal `500.0` matches the `double` column type. Comparison operators check value types at runtime, so `t.price > 500` compiles but throws. See [comparison operators](#).

Upsert several rows

The batch `upsert` inserts and updates rows in a single atomic operation, so no rows are written if any row fails:

```
var products = await Product.db.upsert(
  session,
  [
    Product(sku: 'chair-01', name: 'Office chair v2', price: 249.00),
    Product(sku: 'desk-01', name: 'Standing desk', price: 599.00),
  ],
  conflictColumns: (t) => [t.sk],
);
```

The batch method takes the same `updateColumns` and `updateWhere` parameters as `upsertRow`. The result contains one row per input, in the same order. When `updateWhere` is set, conflicting rows that do not match are skipped and left out of the result, so the list can be shorter than the input. For large batches, the read-back can be skipped entirely. See [Skipping returned rows](#).

Like the other batch operations, `upsert` accepts a `transaction` parameter to join a larger [transaction](#). For models with [non-persistent fields](#), the input values of those fields are carried over to the returned objects. They take no part in conflict detection and are never written to the database, and such batches are upserted row by row internally, which can be slow for large inputs.

A single-row upsert that unexpectedly matches multiple rows throws a `DatabaseUnexpectedResultException`. See [exceptions](#).

Delete

Deleting rows from the database is done in a similar way to updating rows. However, there are three delete operations available.

Delete a single row

To delete a single row, use the `deleteRow` method.

```
var company = await Company.db.findById(session, companyId); // Fetched company has its
id set
var companyDeleted = await Company.db.deleteRow(session, company);
```

The input object needs to have the `id` field set. The `deleteRow` method returns the deleted model.

Delete several rows

To batch delete several rows, use the `delete` method. This method also supports [ordering](#) the returned deleted results.

```
var companiesDeleted = await Company.db.delete(
  session,
  companies,
  orderBy: (t) => t.id, // or t.id.asc()
);
```

This is an atomic operation, meaning no entries will be deleted if any entry fails to be deleted. The `delete` method returns a `List` of the models deleted, ordered as specified by the `orderBy`.

Delete by filter

You can also do a [filtered](#) delete and delete all entries matching a `where` query, by using the `deleteWhere` method. This method also supports [ordering](#) of the returned deleted results.

```
var companiesDeleted = await Company.db.deleteWhere(
  session,
  where: (t) => t.name.like('%Ltd'),
  orderByList: (t) => [t.name.desc(), t.id.asc()],
);
```

The above example will delete any row where the `name` ends in *Ltd*. The `deleteWhere` method returns a `List` of the models deleted, ordered by name in descending order, followed by id in ascending order.

Skipping returned rows

The batch and filtered write methods read the affected rows back from the database and return them as a list. Pass `noReturn: true` when the write itself is all you need.

```
await Company.db.updateWhere(
  session,
  columnValues: (t) => [t.name('Archived company')],
  where: (t) => t.name.like('%Ltd'),
  noReturn: true,
);
```

The write runs with the same filters, conflict handling, transaction, and atomicity guarantees, but the method returns an empty list instead of reading the rows back. Skipping that read saves transferring and deserializing every affected row, which is worth it for bulk imports, cleanup jobs, and any write where the generated ids, database defaults, and updated values are not used afterwards.

The `noReturn` parameter is available on `insert`, `update`, `updateWhere`, `upsert`, `delete`, and `deleteWhere`. The single-row methods `insertRow`, `updateRow`, and `deleteRow` always return the affected row. `upsertRow` returns it too, or `null` when the existing row doesn't match its `updateWhere` expression.

Since the result is empty, `orderBy` and `orderByList` have no visible effect on it. They still matter on filtered operations, where they decide which rows a `limit` or `offset` selects.

Count

Count is a special type of query that helps counting the number of rows in the database that matches a specific [filter](#).

```
var count = await Company.db.count(  
  session,  
  where: (t) => t.name.like('s%'),  
);
```

The return value is an `int` for the number of rows matching the filter.